The goal of my first Capstone project mathbot installation will be to service a middle school math class of about 30 students over 30 days, to create good arithmetic math conversations for that class. If such a proof-of-concept period is productive, and if the deployment plan that I formulate (briefly below and on the next page) is flexible enough to grow and to service greater numbers of students in the future, a path forward will have been found.

A crucial initial requirement for deployment of my math chatbot is to function as a service (a daemon basically) that can converse with 30 students at a time, recording logs for all student interactions in a manner that a teacher can examine individually, storing all student logs with timestamps and student names. Those student logs will be used to guide an understanding of future requirements in a 2.0 version of the math bot. Another crucial requirement is that a math teacher must be able to create JSON files as input for the math chatbot, and start-and-stop the service at any time.

In a public-school environment, deployment flexibility is also a big feature. Even when particular hardware or a particular local network is down, students still should be able to run the chatbot at all hours of the day.

The chatbot needs to create a Python server able to handle 30 non-blocking connections at the same time[1].

In order for the chatbot to be deployable to any platform, the chatbot needs to be in a Docker container. And in order to manage the deployment details of the Docker container to allow 24-hour-uptime usually over 7 days a week, I need to manage redundant deployment replicas within Kubernetes including storage-and-retrieval of secret API keys for OpenAI and Gemini[2].

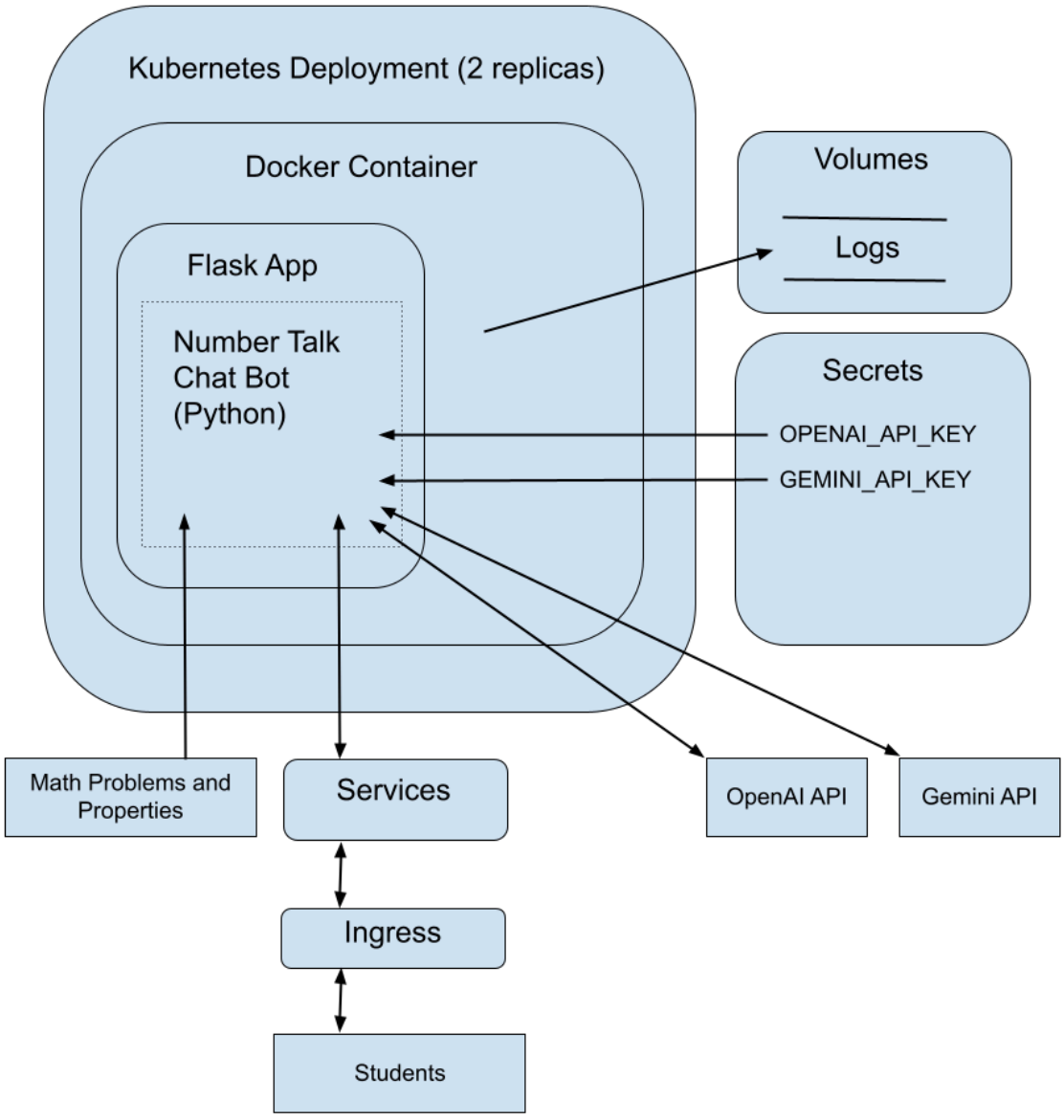
My plan is to deploy the math chatbot in a Flask server, written in Python, which automatically creates a new thread for each connection via a web browser that students make. The Flask server will have a GET endpoint to ask math questions, and a POST endpoint to receive student inputs. None of the endpoints will be idempotent PUT endpoints, which is actually important because the actual *history* of answers from a student matters a lot for this application.

The Flask server will exist in a Docker container whose OPEN_API_KEY and GEMINI_API_KEY are managed as secret keys in a Kubernetes .yaml file. For the purpose of this project, a single-use Kubernetes minikube will be used containing one docker container, with the assumption that by extending and re-configuring the Kubernetes .yaml file(s), more heavy-weight future installations can evolve from this starting path. A rough initial deployment diagram is shown on the next page.

---

[1] Perhaps using the Python “asyncio” library. It needs to use the Python library “fastapi” to create a web server with “start,” “stop,” and “reconfigure” commands

[2] At this time these deployment details are still vague to me, because I lack a solid DevOps and CI/CD background. One of the exciting things about this course is that the course forces me to use these tools in a detailed manner in order to deploy, log, collect statistics on, potentially further train, and manage my capstone project math chatbot.


Kubernetes Deployment (2 replicas)
Docker Container
Flask App
Number Talk
Chat Bot
(Python)
Volumes
Logs
Secrets
OPENAI_API_KEY
GEMINI_API_KEY
Math Problems and Properties
Services
OpenAI API
Gemini API
Ingress
Students